

Multi-Channel Hebbian Plasticity in Multilayer BDH Solves Context-Conditional Binding by Partitioning Memory

A window gate, slow memory and a targeted channel split find an eight-way partition without labels or restarts, on two machines with disjoint seeds

A technical report • Revision 8 (draft)

Project repository: [Ynkling/clankers](#)

Abstract

Revision 7 left eight streams unsolved: no learned-gate run trained on all eight from the start had bound, and screens traced the failure to the gate’s recurrent state, which stops carrying the context before any routing forms. This revision replaces the gate. A *window gate*, a width-3 causal convolution of the token embeddings with no recurrence, was screened in the first exploratory batch, beat the recurrent gate there, and was dropped because it cannot see a distant cue. An audit of the project’s records brought it back. Combined with slow memory for the first 2400 updates and a plateau-triggered *channel split* that copies the busiest channel’s gate row onto the idlest, it binds eight streams with one channel per stream in 38/40 runs of a pre-registered test, against 0/40 for the recurrent-gate recipe (38 vs 0 discordant pairs, $p = 3.6 \times 10^{-12}$) and 15/40 for the window gate without the split (23 vs 0, $p = 1.2 \times 10^{-7}$). The two machines ran disjoint seeds, so the pooled test is a test. The recipe uses no labels, no restarts and no hinge; its reading, fixed before the run, was “eight streams bind without labels or restarts”. At four streams with sixteen channels the window gate binds 38/40 against 27/40 for the recurrent gate (13 vs 2, $p = 0.004$); at two streams it is within the pre-registered bound of the previous recipe, and better by two pairs.

Three exploratory looks at eight streams, on 30 seeds (one of them the test’s own printed control), agree that the split’s row copy is its active part and that the optimizer reset bundled with it adds nothing. Two anti-collapse devices borrowed from the literature did not help: Gumbel noise on the read gate collapses routing and on the write gate worsens eight-stream merges, and a load-balancing loss changed the shape of the routing at four streams without changing its count. At four streams the recipe’s constants survive a fourfold change of width and a doubling of depth without retuning, on ten seeds per size. Muon is not an ingredient: the recipe binds eight streams at 9/10 under either optimizer, and a pooled test of Muon’s four-stream speed, shown on one machine in Revision 7, was not shown. None of twelve papers reviewed carries an inferred context cue across distance without labels; our own distant-cue task remains unsolved, and screens found that a single memory channel with a delta-rule write binds the two-stream header layout by routes that need no partition (context-tagged keys at a fixed decay; a learned forget gate that partly clears at the context token), so the partition’s claim is about the number of contexts: at eight streams the gated recipe binds 38/40, and no single-channel run has yet been made there. We correct several of our own statements, among them that the gate “needs only one token of memory” and that Muon “binds four streams sooner”.

Revision 8 (draft). This revision adds Phase VI: two pre-registered tests run on two machines with disjoint seeds, an audit of the project’s records ([docs/audit_2026-10.md](#)), five exploratory batches on the side branch, three further exploratory sessions run in parallel on their own branches, and a reading of twelve papers on context routing and memory ([docs/reading/](#)). Two machines ran every main-line test: **X**, a cloud container on Intel Xeon hosts at 2.10 and 2.80 GHz, and **L**,

an Intel i7-12650H. From the Muon-recipe test on, X and L run disjoint seeds, and the pooled claim is primary. The exploratory sessions are **E** (branch `claude/outside-ideas`, batches 1–19), and **F**, **G** and **H** (branches `claude/explore-F/G/H`); F and H ran on 2.10 GHz Xeons; E’s batches ran mostly on 2.10 GHz Xeons, with batch 15’s first segment, most of batch 16 (every Muon run among them) and batch 19 on 2.80 GHz hosts. Every verdict was fixed before its run and printed mechanically. The revision is a draft: E’s batch 19 is running, a follow-up to F and a session G on the distant-cue task were launched after the runs reported here, and everything remains one small model on a synthetic task.

test or batch	question	verdict	§
muon_recipe (X, L, pooled)	the early hinge window under Muon; Muon’s speed at four streams and eight keys	M1 SHOWN pooled (and on L); M2, M3 NOT SHOWN pooled; reading not met	7
window_gate (X, L, pooled)	the window gate with slow memory, no hinge, at 2, 4 and 8 streams; the split at 8	G1, G2, G4 SHOWN pooled; G3 HOLDS; reading “eight streams bind without labels or restarts”	5
batch 15 (E)	a cap on the recurrent gate’s gain	no arm bound: not	8
batch 16 (E)	the window gate with slow memory at 2, 4, 8 streams; a reservoir gate; eight-stream knobs	20/20 at four streams; 3/10 at eight, the first ever	4
batch 17 (E)	the targeted split on the window gate at eight streams	9/10 vs 3/10 (6 vs 0, $p = 0.031$)	4
batch 18 (E)	reset-only control; Muon; the split at $k = 4$; two-stream key splits	copy does the work; recipe carries to Muon; key splits unresolved	6, 9
batch 19 (E)	width and depth	scale-free to $N = 1024$, $D = 128$, 4 layers	10
session H	Gumbel noise, balance loss, reset-only at 4 and 8 streams; gate temperature	nothing replaces the split; the copy, not the reset	6
session F	delta-rule channels; learned forget gates; the header layout	a single delta channel binds the two-stream header layout; worse at eight streams	11

Table 1: Phase VI. Main-line tests are files `test_<name>.py` whose docstring records the pre-registered design and, after the run, the result on each machine and pooled; their specifications are committed to `specs/` before the test is written. Exploratory batches and sessions write only new files on their own branches and label every output “exploratory, not a result”. Appendix A lists the commits.

1 Background

BDH [1] stores working memory in synaptic state updated by a Hebbian rule. In its GPU form the operation is linear attention: the score between a query position t and a past position s is a content match $x_t \cdot x_s$ between sparse positive neuron activations, weighted by a positional operator (scalar decay or RoPE). The public implementation, `bdh.py`, ties queries to keys ($Q = K$), shares one layer’s weights across depth, and LayerNorms the attention output. Its state update, in the paper’s notation, is $\rho_t = \rho_{t-1} + \text{LN}(Ey_t) x_t^\top U$: a pure additive write.

The extension gives every synapse k channels and every token a gate $g_t \in \Delta^{k-1}$ that distributes its write over the channels and blends its read from them. With one distribution for both, the

effective score becomes

$$\underbrace{(x_t \cdot x_s)}_{\text{content match}} \times \underbrace{(g_t \cdot g_s)}_{\text{context-gate match}} \times \text{decay}^{t-s}, \quad (1)$$

strictly causal ($s < t$), so tokens sharing a context can read each other’s memory while tokens in different contexts are isolated. The score is the three-factor form of Triadic Linear Attention [3] with g as the second key; there the second key is a learned projection of the token, here it is a routing distribution inferred from context, and the subject is its label-free discovery. With $k = 1$ the model is single-channel BDH. One gate g_t is computed per token from the raw token embedding v_t (not LayerNormed, not convolved, not the residual stream) and applied at every layer. The hand-set *perfect gate* sends every token of stream s to channel s ; it is used only as a validity ceiling.

Two learned gates. The *recurrent gate* of Phases II–V,

$$h_t = \tanh(W_{\text{in}} v_t + W_h h_{t-1}), \quad g_t = \text{softmax}(W_g h_t), \quad (2)$$

with 32 units in h_t and $W_{\text{in}}, W_h, W_g \sim \mathcal{N}(0, 0.1^2)$ (so that $\rho(W_h) \approx 0.6$ and $\sigma_{\max}(W_h) \approx 1.08$ at initialization). The *window gate* of this revision,

$$h_t = \tanh\left(W_{\text{in}} \sum_{j < 3} w_j \odot v_{t-j}\right), \quad g_t = \text{softmax}(W_g h_t), \quad (3)$$

a depthwise causal convolution of width 3 over the embeddings (weights drawn uniformly in $\pm 1/\sqrt{3}$, no bias), with no recurrence. It sees the current token and the two before it, which in the grouped layout is enough to see each key’s stream token at the key, the value and the query. It cannot see anything further back, and it has no notion of position.

The uniform gate is a stationary point. Writing each gate as a deviation from uniform, $g_t = \frac{1}{k} \mathbf{1} + \delta_t$, gives $g_t \cdot g_s = \frac{1}{k} + \delta_t \cdot \delta_s$. The constant is cancelled by the LayerNorm after attention, so routing enters only through products of deviations, and at the uniform gate the task loss exerts no first-order pull toward any routing. This is why discovery is a race between the gate’s first commitment and whatever the memory learns first, and why the recipes below act in the first few thousand updates.

A short causal convolution. Most arms include the short convolution that Mamba and most linear-attention models place before their token mixer [4]: a depthwise causal filter of width 4 at every layer, initialized to the identity, feeding the queries and the values. In the two-stream configuration without it, the perfect gate failed to bind on the four seeds tried in batch 18 and session H, while session F found it binding 10/10 on ten other seeds: seed-dependent, which is itself a sign that the convolution carries part of the binding, not only the gate’s input.

2 Earlier phases in brief

Phases I–IV (Revisions 1–6). A gate that was a function of token identity could not express the routing (Phase I); a recurrent gate could (Phase II). On the binding task of Section 3, two channels with the perfect gate bind two streams where no single-channel model binds, and a label-free learned gate finds the routing on about half its runs (Phase III). A restart rule made two streams reliable; four streams with four channels mostly failed by merging streams into a shared channel, and spare channels removed most merges (Phase IV).

Phase V (Revision 7). An early-training recipe replaced restarts at two streams and at four with sixteen channels: for the first 2400 updates the memory trains at a tenth of the gate’s rate (**SLOW**), and a hinge penalizes the gate when its channel choice at key positions is explained by position (**HINGE**; **WINDOW** switches the hinge off after update 2400). Eight streams failed in every arm (none of more than 100 runs), and screens located the failure in the recurrent gate: its state carries the stream at initialization and loses it within the first 600–1200 updates, as its recurrent term grows and, in seven of ten runs, the spectral radius of W_h rises above one. Revision 7 ended with a cap on that recurrence being screened.

3 The task, the outcomes and the recipes

Task. There are S streams, each with a context token CTX_s . For each of P keys, S distinct values are drawn from 16 value tokens, so every key is bound to a different value in every stream. In the *grouped* layout the body is SP triples $[\text{CTX}_s, \text{KEY}_i, \text{VAL}_{i,s}]$, grouped by key, keys in random order and stream order random within each key’s group. One query $[\text{CTX}_q, \text{KEY}_q, ?]$ ends the sequence; the model predicts the value from the query’s KEY position, where CTX_q is one token back. The write that must carry the stream is at the VAL position, where CTX is two tokens back. Chance among values is $1/16$; a model that binds keys to values but cannot tell streams apart scores $1/S$. The *header* layout, used in screens, gives each stream one block $[\text{CTX}_s, \text{KEY}, \text{VAL}, \text{KEY}, \text{VAL}, \dots]$ with the stream token once at the start, so most writes are far from their cue. The model has three layers, decay 0.95 (half-life about 14 tokens), $N = 256$, embedding width 32 and one head, and trains at batch 32 for a budget set per test (24,000, 28,800 or 43,200 updates), with evaluation every 1200 updates on 2048 held-out queries; a run stops early after three consecutive evaluations at or above 0.95.

Outcomes.

- **Bound:** an evaluation reaches 0.95 and every later one stays there; the *transition* is the first such step. A run that reaches its budget counts as bound on a single final evaluation; this has happened in 11 of 1028 bound runs in the project’s records.
- **Discovered** ($k = S = 2$): bound, with the streams’ read gates separated at value positions.
- **Bound routed** ($k \geq S > 2$): bound, every stream has its own channel (a one-to-one *stream-to-channel map*, measured at value positions, where the writes are), and every stream’s accuracy is at least 0.9.
- **Failure classes:** POSITION or KEY when the gate’s channel choice at key positions is explained ($\eta^2 \geq 0.5$) by position or by key identity; STREAM-PARTIAL; OTHER. For $S > 2$: MERGED (j share) when j streams share a channel, non-stream when the gate does not split by stream, *collapsed* when final accuracy is below 0.15.

Recipes.

- **SLOW:** one Adam optimizer with two groups. The gate’s parameters train at 10^{-3} throughout; every other parameter, embedding and convolution included, at 10^{-4} for updates 1–2400 and 10^{-3} after.
- **HINGE, WINDOW:** Revision 7’s position hinge, on the recurrent gate; kept here only as the comparison.

- **SPLIT (KEYMASS):** at checks every 2400 updates from 4800, on a fixed 64-sequence probe, if the probe’s accuracy is below 0.95 and rose by less than 0.02 since the previous check, copy the gate row $W_g[c^*]$ of the channel with the largest mean read-gate mass at key positions onto the row $W_g[c_0]$ of the channel with the smallest, add noise of 0.1 times the row’s standard deviation to both, and zero W_g ’s Adam state; at most three splits per run, at least 4800 updates apart. It uses no stream labels; it needs to know where the key positions are.
- **MUON:** Muon at learning rate 0.005 on every 2-D weight (momentum 0.95, Nesterov), with Adam on the rest, and SLOW’s schedule.

Statistics. Paired comparisons in the main-line tests use exact one-sided McNemar tests on the discordant pairs (“ b vs c ”), unpaired ones Fisher’s exact test; E’s exploratory batches report two-sided McNemar p -values and session H one-sided ones, as marked where quoted; bands are RELIABLE at $\geq 90\%$ and MAJORITY at $\geq 50\%$ of runs, with Wilson 95% intervals. From the Muon-recipe test on, X and L run disjoint seeds (X 400–559, L 1400–1559), the pooled claim over both machines’ pairs is primary and each machine’s is secondary. About sixty pre-registered one-sided claims have been made per machine at $\alpha = 0.05$ over the project, so two or three false positives are expected under the global null; the conclusions that matter below are those replicated on both machines or pooled over disjoint seeds.

4 The window gate

History. The window gate was the first screen of the first exploratory batch (S1, batch 1): on seeds 160–179 with plain Adam and no recipe it discovered 15/20 against the recurrent gate’s 7/20 and bound faster; its five failures were four key splits and one run routed but not bound (it cannot position-split, having no position). Batch 2 ran it on the header layout, where it bound 2/10, both by key splits, and it was dropped: a gate that cannot see a cue three tokens back seemed the wrong direction for a language model. It was never combined with slow memory and never run at four or eight streams. The October audit of the project’s records (`docs/audit_2026-10.md`, Section 3.1) noticed that every obstacle of Phase V was a property of the recurrent parameterization: position splits are the recurrence counting triples, and the eight-stream loss of the stream is the recurrence becoming expansive. A window gate can do neither. The audit fixed a prediction before any run: with slow memory, at two streams $\geq 36/40$ with key failures gone; at eight streams, binding on some seeds where every recurrent arm bound none.

Batch 16 (E). The window gate with SLOW and no hinge (LOCAL3_SLOW): at two streams 31/40 on seeds 160–199 (STREAM-PARTIAL 5, KEY 4, no POSITION), inconclusive against the hinge recipe’s 34/40 (5 vs 8). At four streams with sixteen channels, 20/20 bound routed against X’s recorded HINGE4k16 15/20 (5 vs 0, two-sided $p = 0.0625$). At eight streams, 3/10 under Adam and 3/10 under Muon, where every recurrent arm in batches 13–16 had bound 0/10; the seven unbound Adam runs ended with two or three streams sharing a channel at accuracy 0.73–0.88. The window gate’s state decodes the stream at both key and value positions (1.00 at four streams from update 4800; at eight, 0.93–1.00 under Muon and, under Adam, 0.55–0.67 at updates 2400–4800 recovering to 0.87–0.88 by 9600), where the recurrent gate’s reads 0.14–0.28. So the audit’s first prediction was partly met (no position failures; key failures reduced, not gone, and 31/40 short of 36), and its second was met.

Batch 17 (E). The failures at eight streams were merges, so batch 17 added the targeted split of Section 3 (S37’s KEYMASS rule, at most three splits). W_SPLIT bound routed 9/10 against 3/10 for the same seeds without it (6 vs 0, two-sided $p = 0.031$); every one of its 19 splits was on target by the labels (the busy channel held two to five streams, the idle one none), and in six of nine bound runs the transition came at the evaluation right after the last split. Decay 0.98 alone bound 1/10 (it settled the merges early) and a longer budget alone 4/10: the splits, not the decay or the time, do the work. The screen handed W_SPLIT back for a pre-registered test.

5 The main result: test_window_gate

The test’s specification was committed before the test was written (`specs/test_window_gate.md`) and amended, before any Part D run, to add the split. Both machines ran it on their own seeds (X 500–559, L 1500–1559). Part A ($S = 2$, $P = 4$, $k = 2$, no convolution, 24,000 updates, 40 seeds per machine): the window gate with one learning rate (WIN3), with slow memory (WIN3_SLOW), and the recurrent-gate recipe (HINGE0). Part B ($S = 4$, $P = 4$, $k = 16$, convolution, 28,800, 20 seeds): WIN3_SLOW16 against WIN16_A, the recurrent gate under Adam with the early hinge window. Parts C and D ($S = 8$, $P = 4$, $k = 16$, convolution, 43,200, 20 seeds): WIN3_SLOW_D8, HINGE_D8 (the recurrent recipe) and WIN3_SPLIT_D8 (the window gate with the split), with a reset-only control (WIN3_RESET_D8: the Adam-state reset at the same trigger, no copy) on five seeds per machine, printed and not claimed. The perfect gate on two seeds per part was the validity arm, and bound on every one (Part C at 4800 and 3600–6000). Every CHECK passed on X, including bit-for-bit reproductions of batch 1’s window gate, batch 16’s LOCAL3_SLOW and batch 17’s W_SPLIT (its split at 4800 included); L’s file carries the metadata the test writes only after its verification passes. Nothing was cut; X ran its 225 learned-gate records in 30.9 h of training (its six perfect-gate records came from an earlier start that a container restart ended), L its 231 records in one start (projected 17.3 h).

What the runs show (diagnostics, not claims). The recurrent-gate recipe bound none of 40 eight-stream runs (15 collapsed, 17 non-stream, 8 merged). The window gate alone bound 15/40, and every one of its 25 failures was a merge of two, three or four streams on one channel. The split raised that to 38/40: the trigger fired 85 splits in 40 runs (none to three per run), every target labelled on target, and brought the median time to bind forward (X: 21,600 against 27,600; L: 18,600 against 32,400; medians over bound runs). The two failures (X’s seeds 555 and 556) ended with two streams sharing a channel. The reset-only control matched the window gate without the split seed for seed on L (3/5 and 3/5, 0 vs 0 discordant) and bound 0/5 against 1/5 on X: on these seeds the row copy, not the Adam reset, made the difference (Section 6). At four streams, WIN3_SLOW16’s two failures were one unrouted bind (X) and one merge (L); WIN16_A’s thirteen were four position splits, three merges, four unrouted binds and two others. At two streams no arm failed by position; WIN3’s failures were key splits (12 of 15), WIN3_SLOW’s key splits and partial streams (6 and 5 of 11), and HINGE0’s key splits (10 of 13), with two OTHER and one partial. Slow memory on the window gate at two streams is printed, not claimed: 69/80 against 65/80 (7 vs 3, $p = 0.17$).

What is and is not established. The result is a pre-registered, two-machine, disjoint-seed test of a recipe that uses no stream labels, no restarts and no hinge, and that finds an eight-way partition of the memory with one channel per stream in 38 of 40 runs where the previous recipe found none. The split needs to know where the key positions are (as the hinge did); the window gate needs the cue within two tokens; and the task is the grouped layout, where it is. Neither limitation is new.

part	arm	X	L	pooled
A: $S=2$, $P=4$, $k=2$, discovered	WIN3	33/40	32/40	65/80
	WIN3_SLOW	33/40	36/40	69/80
	HINGE0 (recurrent, Rev. 7’s recipe)	33/40	34/40	67/80
B: $S=4$, $k=16$, bound routed	WIN3_SLOW16	19/20	19/20	38/40
	WIN16_A (recurrent, early window)	15/20	12/20	27/40
C, D: $S=8$, $k=16$, bound routed	WIN3_SPLIT_D8 (the recipe)	18/20	20/20	38/40
	WIN3_SLOW_D8 (no split)	6/20	9/20	15/40
	HINGE_D8 (recurrent, Rev. 7’s recipe)	0/20	0/20	0/40
	WIN3_RESET_D8 (control, of 5)	0/5	3/5	3/10
	perfect gate (of 2)	2/2	2/2	4/4

Table 2: test_window_gate. Pooled claims (primary, exact one-sided McNemar over both machines’ pairs): G1, WIN3_SLOW16 beats WIN16_A: 13 vs 2 (X 5 vs 1, L 8 vs 1), $p = 0.0037$, SHOWN. G2, WIN3_SPLIT_D8 beats HINGE_D8: 38 vs 0, $p = 3.6 \times 10^{-12}$, SHOWN. G3, WIN3_SLOW not worse than HINGE0 by more than two discordant pairs: HINGE0 only 9, WIN3_SLOW only 11, $d = -2$, HOLDS (a bound, no p -value). G4, WIN3_SPLIT_D8 beats WIN3_SLOW_D8: 23 vs 0 (X 12 vs 0, L 11 vs 0), $p = 1.2 \times 10^{-7}$, SHOWN. Per machine every claim is SHOWN or HOLDS on both, except G1 on X (5 vs 1, $p = 0.11$; on L 8 vs 1, $p = 0.020$). Bands, pooled: WIN3_SPLIT_D8 RELIABLE, Wilson [0.835, 0.986]; WIN3_SLOW16 RELIABLE [0.835, 0.986]; WIN3_SLOW MAJORITY [0.770, 0.921]; WIN3_SLOW_D8 MINORITY [0.242, 0.530]; HINGE_D8 NEVER [0, 0.088]. The pre-registered reading, “eight streams bind without labels or restarts” (WIN3_SPLIT_D8 RELIABLE pooled and G2, G4 SHOWN), applies.

The audit’s over-prediction for two streams (“ $\geq 36/40$ with KEY failures gone”) was not met on X (33/40) and KEY failures were reduced, not removed; what was claimed, G3, holds.

6 The split: the copy, not the reset

The split bundles two actions, a row copy and an optimizer-state reset, and Revision 7’s screens had no reset-only control (audit, Section 2.3). Phase VI ran four looks at it, on disjoint seeds and with two different implementations of the reset.

where	configuration	split	reset only	discordant, p
batch 18, S50 (E)	$S=8$, $k=16$, 260–269; the Adam-state reset replayed at the recorded split updates	9/10	2/10	8 vs 1, two-sided $p = 0.039$
session H, S58b	$S=8$, $k=16$, 460–469; live trigger	10/10	2/10	8 vs 0, one-sided $p = 0.0039$
window_gate control	$S=8$, $k=16$, 540–544 and 1540–1544; live trigger	—	3/10	vs no split 4/10 on the same seeds, 0 vs 1
session H, S58	$S=4$, $k=16$, 420–459	39/40	39/40	0 vs 0; uninformative (no split needed: NONE 38/40)

Table 3: The split’s two actions. In the eight-stream looks the reset-only arm fired under the live rule (S58b, the test’s control) or at the recorded split updates (S50), on targets labelled on target where labelling was recorded (S58b, X’s control), and, where a no-intervention arm exists (S50, the test’s control), bound no more often than it. At four streams with sixteen channels nothing is needed, so S58 cannot tell the actions apart.

Pooled over the two ten-seed screens, which differ in budget and in how the reset was applied, 19/20

against 4/20. The copy puts the idle channel’s gate logits in a near-tie with the busy channel’s, so the next gradient step can separate two merged streams; a reset leaves the idle channel where it was. “The copy, not the reset” remains printed, not claimed; it will be a pre-registered claim in the next main-line test.

What does not replace it (session H). Two devices the literature uses against router collapse were run beside the split and the reset at four streams, $k = 16$ (40 paired seeds), where every arm but one sat at the ceiling (NONE 38/40) and no reading could apply; a rule fixed beforehand then carried the two best-scoring arms, which were the reset and write-gate noise, to eight streams (10 seeds). Gumbel(0, 1) noise on the gate’s logits during training, Raven’s sole anti-collapse device [13]: on both the read and write gates it collapsed routing even at four streams (33 of 40 runs merged), consistent with our read being routed where Raven’s is dense; on the write gate alone it bound 37/40 at four streams with a later transition (median 6000 against 3600) and a lower margin, and at eight streams it made the merges worse (0/10, with up to six streams sharing a channel; 10 vs 0 against the split). A Switch-style balance loss, as Mixture-of-Memories uses [12, 17]: 36/40 at four streams, routed by stream, but in 34 of those 36 runs the streams were mapped to different channels at key positions than at value positions and the routing margin fell from 0.87 to 0.46; it changed the routing’s shape, not its count, and was not carried to eight streams. At two streams ($k = 2$, no convolution, ten seeds), a learned gate temperature with a floor ran to the floor in every run, and a stable-max softmax changed nothing: the gate wants to be sharper, not softer, and in those runs a saturated gate marked a committed wrong partition, not a plateau before binding.

A guard the split needs at $k = S$. With no spare channel (batch 18, S51, $S = k = 4$), the trigger once fired on a correctly routed map, found no empty channel, overwrote a live one and merged the run; in seven of its eight bound runs it never fired. Spare channels remain the default (Phase IV), and the trigger should not fire when its target channel already carries key mass.

7 Muon is not an ingredient

Revision 7 reported, from one machine, that under Muon the recipe “binds four streams sooner”. The Muon-recipe test (`test_muon_recipe`, the first on disjoint seeds) asked three questions. M1, the early hinge window under Muon at two streams: WIN_M 79/80 against SLOW_M 69/80 pooled, 10 vs 0, $p = 0.001$, SHOWN (on L 6 vs 0, SHOWN; on X 4 vs 0, $p = 0.0625$). M2, Muon binds four streams by update 4800 more often than Adam: 12 vs 8 pooled, $p = 0.25$, NOT SHOWN; per machine it was shown on X (7 vs 1, $p = 0.035$) and reversed on L (5 vs 7), so Revision 7’s one-machine result did not replicate on the other machine’s seeds. M3, the same at eight keys: 7 vs 5 over 30 pairs (X’s time rule cut its Adam arm to ten seeds), $p = 0.39$, NOT SHOWN. The test’s reading, “the Muon recipe is reliable at the working configurations”, was not met: WIN16_M was MAJORITY pooled (33/40). Batch 18 then ran the window recipe with the split under Muon at eight streams: 9/10, the same as under Adam (1 vs 1), and 9/10 against a fresh Muon reference without the split at 2/10 (7 vs 0, two-sided $p = 0.016$). The eight-stream result is the recipe’s, not the optimizer’s. Muon’s bf16 Newton–Schulz step does not reproduce across processors, which cost the project paired comparisons; nothing in the main line now needs it.

8 The recurrent gate’s eight-stream failure, closed

Revision 7’s reading was that the recurrent gate’s state stops carrying the stream when its recurrence becomes expansive, and it left a cap on that recurrence being screened. Batch 15 (S41) ran the cap at 0.5 on the spectral norm of W_h under Muon, alone, with a previous-token input, and with the previous-token input and no recurrence, and the capped previous-token input under Adam: the three Muon arms bound 0/10 and the Adam arm 0/5. With the look-back the capped gate’s state kept the stream at key positions throughout (decodability 0.93–1.00) and still ended with the eight streams on three to seven channels; final accuracy rose in every pair (median 0.61 against 0.37) but no run bound. Batch 16 added a frozen reservoir recurrence (0/10), decay 0.98, a tenth of the rate on W_h , and a higher learning rate: none bound (the perfect gate binds at decay 0.98, so the decay is valid). The window gate then bound where all of these had not. Two corrections to Revision 7 follow: the gate’s state must carry the stream at the *write* position, where CTX is two tokens back, not only at the read, where it is one back, so the sentence “the gate needs only one token of memory” was wrong (the audit noted it; batch 16’s S47 measured the write positions, which S38–S41 had not); and S41’s no-recurrence arm, whose input is (v_t, v_{t-1}) , cannot by construction see the stream token at value positions, so its ten key splits were a property of that window and not evidence about recurrence (audit, Section 2.3). The cap, as specified, also acted from update 1 ($\sigma_{\max}(W_h) \approx 1.08$ at initialization), not only after the crossing. The diagnosis stands as a description of the recurrent gate; the remedy was to remove the recurrence.

9 Two-stream key splits

At two streams with two channels and no convolution, the window gate fails by routing on key identity: its key splits form within the first 600 updates, while the memory is still at 10^{-4} (batch 17, S49). Slow memory reduces them (WIN3 KEY 12 of 15 failures pooled; WIN3 SLOW KEY 6 of 11), and batch 18 screened two further devices on seeds 160–199: two spare channels ($k = 4$: 36/40, with four key splits still formed by update 325) and the hinge’s key term on updates 1–2400 (36/40 discovered, one KEY failure, but three “STREAM-PARTIAL” failures at exactly $\eta^2 = 0.5$ by key, the signature of a hard two-of-four key pattern, which relabelled would make the reading “neither”). Neither is carried to the main line. The classifier’s handling of that boundary case is to be fixed before any two-stream claim is read again. The configuration is the project’s hardest relative to its size and the one with the least headroom (65–69/80 for every arm); it is also the only one where the window recipe’s margin over the hinge recipe is small (two pairs pooled) and where the hinge recipe won a screen (batch 16: 34/40 against 31/40 on seeds 160–199).

10 Scale

Batch 19 (E, running) asked whether the recipe’s constants are tuned to $N = 256$, $D = 32$, three layers. At four streams with sixteen channels (seeds 270–289, the oracle binding at every size and depth), with every constant unchanged: $N = 512$ and 1024 at $D = 32$, and $D = 64$ and 128 at $N = 256$, bound routed 10/10 each against the reference’s 9/10, all with median transition 3600; 2, 4 and 6 applications of the shared layer bound 10/10, 10/10 and 9/10 against 10/10 for three, with six layers slower (median 4800). Both readings were “scale-free” by their pre-fixed rule (every $d \geq -1$), at four streams only; with ten seeds the screen rules out a large drop, not a small one. Eight streams and larger key loads at the larger widths are running.

11 Distant cues and the memory rule

What the literature holds. Twelve papers on routing, memory and context were read in full (`docs/reading/`): the test-time-regression framework [5], Gated DeltaNet [6], the Tiny Recursive Model and recurrent-depth language models [7, 8], Flesch et al.’s Hebbian context gating [9], Active Dendrites [10], Context-Gated Associative Retrieval [11], Mixture-of-Memories [12], Raven [13], a study of content-based routing [14], the Hierarchical Multiscale RNN [15] and Nested Learning [16]. None of the twelve carries an inferred context cue across distance without labels: each hands the context over (a task identifier, a prototype readable off every input, a clamped context layer), carries it with softmax attention, or routes by token type. The study of content-based routing is the sharpest statement of why our header-layout runs failed: routers fed raw embeddings, recurrent summaries or pooled summaries reach 0.9–29% even with the correct route supervised, and 98–99.7% once an upstream, jointly trained mixing step has written the context into each token’s representation. Flesch et al. supply the rest: a slow integrator must receive only the cue, slow integration merges interleaved contexts, and inside a block the loss exerts no pressure to use a constant cue. Our window gate reads three raw embeddings; our fading-memory gates pooled everything; our header layout is positionally solvable at fixed block length. Those are the failures the papers predict.

BDH-CQ. Pathway’s follow-up [2] describes its memory as “linear correction rules on S ” with linear attention as the special case $S_t = S_{t-1} + U_\theta(D_t)$, and its reasoning as $H_{r+1} = F_\theta(H_r, S_K)$ over a frozen memory; “dimensions, exact update rules, and implementation details remain proprietary”. Our reading (`docs/reading/`, an inference, not a finding): in the taxonomy of [5] and [16], the rules that are affine in S and read S before writing are the delta-rule family (DeltaNet, Longhorn, Gated DeltaNet, DeltaProduct), and the loop matches the recursive-refinement models [7, 8], of which [8] explains why the frozen memory must enter every step. Neither is a distant-cue mechanism. The 97.4% Sudoku figure sometimes attached to BDH is from Pathway’s blog, on internal data with an unstated protocol, and is not comparable to the Sudoku-Extreme numbers of [7].

Session F: delta-rule channels. Session F replaced the channel write by Gated DeltaNet’s rule, $S_t^c = S_{t-1}^c(\alpha_t^c(I - g_t^c \beta_t \hat{x}_t \hat{x}_t^\top)) + g_t^c \beta_t v_t \hat{x}_t^\top$ with unit keys; at $\beta = 0$ its logits agree with the Hebbian model’s to 10^{-7} , and with the new path disabled it reproduces a recorded run bit for bit. We had predicted that a single delta channel would be the negative control the project lacks, since the rule overwrites a key’s value. It was not: with $\beta = 1$ and the fixed decay, on the two-stream header layout at $P = 4$ a single delta channel bound 8/10 (Hebbian 1/10), and on the grouped layout 4/10 against 0/10; a post-hoc look at one rerun (seed 307) found the keys at the third layer tagged by stream and nearly orthogonal across streams. With a learned forget gate (Raven’s routed decay, or Gated DeltaNet’s) a single delta channel bound the $P = 8$ header layout 9/10 and 8/10 against 3/10 with the fixed decay, and, post hoc, had learned to partly clear its memory at the context token (median α of 0.18–0.26 at CTX in the first layer, 0.72–0.74 in the third, 1.00 at keys and values). Under the window gate, with a learned β , the delta rule was ahead at two streams (36/40 against the recorded 31/40 on other seeds; Fisher $p = 0.11$, a bound by the screen’s rule), equal at four (20/20 against 19/20) and worse at eight (3/10 against 9/10, 0 vs 6, $p = 0.016$), where even the delta perfect gate takes 22,800–28,800 updates against 2400–4800 for Hebbian, at about three times the compute per step. A learned write strength β collapsed toward “do not write” whenever it was the only change. The Hebbian write stays on the main line.

What this changes. At two streams the memory has routes to conflicting bindings that need no partition: make the key context-dependent through the stack, so that one delta memory holds

both (the route Nested Learning’s self-modifying key projections describe [16]; Gated DeltaNet itself overwrites), or clear the memory at the context switch (Gated DeltaNet’s $\alpha_t \rightarrow 0$ [6]). So “the partition is necessary” is a claim about the number of contexts, not about two: at eight streams the gated Hebbian recipe binds 38/40, the same recipe on delta channels stalls at 3/10, and a single delta channel has not yet been run. That premise test, one delta channel at four and eight streams on the grouped task, is the next screen. If the one-seed observation of stream-tagged keys at the third layer holds up, the context *is* reaching depth in the header layout, which is what the per-layer residual-stream gate of the audit (Section 3.3) requires; Session G’s first task is to probe it directly.

The distant-cue task. A header layout with random block lengths and eight keys per stream exceeded the oracle’s budget (neither perfect gate bound within 24,000 updates), so Session G has no baseline from it yet; a lighter version with four keys per stream is the next step. What a distant-cue test needs is now clear: random block lengths (so that position cannot route), a budget the oracle meets, and routing reported at write positions, so that “bound” is not mistaken for “routed”.

12 Where the mechanism stands

setting	perfect gate	recurrent gate, Rev. 7 recipe	window gate + SLOW	window gate + SLOW + split	one channel
$S=2, P=4$, no conv.	4/4	67/80	69/80	—	0 of > 200
$S=4, P=4, k=16$, conv.	4/4	27/40 ^w	38/40	39/40 ^e	—
$S=8, P=4, k=16$, conv.	4/4	0/40	15/40	38/40	—

Table 4: The main configurations in `test.window_gate` (pooled over X and L, disjoint seeds; bound routed, or discovered at $S = 2$). ^wWith the early hinge window (WIN16.A); Revision 7’s full-hinge arm bound 35/40 on shared seeds, with routing recorded only on X (15/20). ^eSession H’s S58, exploratory, 40 seeds. The one-channel column is Phase III’s; one channel has not been run at four or eight streams on the grouped layout.

The partition does what one Hebbian channel cannot, and the gate finds it. Given the partition, every grouped-layout configuration tried binds; one Hebbian channel does not, where it was run (two streams at four and eight keys; four streams with four channels). A gate that reads three tokens, trained with the memory held slow for 2400 updates and with a channel split when binding stalls, finds the partition at two, four and eight streams without labels or restarts.

Discovery is a race, and the recipe’s parts each remove one way of losing it. Slow memory keeps the memory from exploiting a non-stream split first; the window gate cannot count positions, so position splits are gone; the split breaks the merges that remain at eight streams by giving an idle channel the busy channel’s logits. The recurrent gate’s pathologies (position splits, the expansive recurrence) were not obstacles to the mechanism; they were obstacles of that gate.

Two routes, and the partition’s claim. A single delta-rule channel binds the two-stream header layout (8/10 at a fixed decay; 9/10 and 8/10 with a learned forget gate at eight keys) and the grouped two-stream layout on 4/10. The partition’s advantage, so far, is in the number of contexts; one channel at eight streams is untested.

13 Retrospective

13.1 Revision 7’s claims

“At eight streams the gate forgets the context” (subtitle). Superseded. True of the recurrent gate; the window gate carries the context at every write position by construction and binds 38/40.

Revision 7’s recipe (slow memory and a hinge) binds without restarts at two streams and at four with sixteen channels. Stands, and is replaced by a recipe without the hinge, which equals it at two streams (within the bound), beats its early-window form at four (G1) and beats it at eight (G2).

“The gate needs only one token of memory” (Section 11 of Revision 7). Corrected. One at the read, two at the write; the write-position decoders were not measured in Revision 7’s screens S38–S41 (batch 16, S47, measured them).

“Under Muon the recipe binds four streams sooner.” Not shown on disjoint seeds pooled (12 vs 8, $p = 0.25$), and the machines disagreed. The one-machine result was a seed effect.

“A cap on the gate’s recurrence is being screened.” Resolved: not. No capped arm bound (0/10, 0/10, 0/10 and 0/5); the cap acted from update 1.

The row copy breaks merges (Section 10 of Revision 7, from S36/S37, which scored BOUND). Supported with the right outcome (bound routed) and a reset-only control, at eight streams, in three exploratory looks; printed, not yet claimed.

13.2 Errors made during Phase VI

Predictions that the data overturned. We predicted that a single delta channel could not bind conflicting streams (it binds the two-stream header layout 8/10); that Gumbel noise on the gate’s logits could replace the split (on the read gate it collapsed routing; on the write gate it worsened eight-stream merges); that a balance loss would attack merges (where it was run there were no merges to attack, and it changed the key- and value-position maps instead); that a learned forget gate would widen the gap between the oracle and one channel (it shrank it, on the eight-key header layout); and, in the audit, that the window gate with slow memory would reach $\geq 36/40$ on seeds 160–199 with no key failures (it reached 31/40 there, and 33 and 36 of 40 on the main test’s seeds, with key failures reduced).

A distant-cue task that exceeded its oracle. Session F’s randomized header layout did not bind under the perfect gate within its budget, so it gave Session G no baseline. The specification fixed the block lengths and left the key load open; F’s resolution, eight keys per stream, matched the eight-key header layout that the oracle already binds slowly.

A classifier at its own threshold. Three batch-18 runs sit at exactly $\eta^2 = 0.5$ by key and are labelled STREAM-PARTIAL; the hard two-of-four key pattern should be KEY. The reading that depended on them is withdrawn until the classifier is fixed.

Attribution of a number. An earlier working note put Pathway’s 97.4% Sudoku figure beside TRM’s 87.4% as if on one protocol; the figure is from Pathway’s blog, on internal data, and the protocols differ.

Process. Container restarts cost in-flight runs on X (the test’s first start), on F (twice, while the session was idle with only detached processes running) and on E in several batches (batch 16’s segments 2, 6 and 7 among them); on F a harness-tracked keepalive stopped it, and some form of it should be in every long run. Every resumed segment began by reproducing a recorded run bit for bit. Batch 18’s Muon reference did not reproduce across hosts and was rerun fresh, as the rules require.

14 Methodological findings

Revisions 5–7 recorded sixteen lessons; Phase VI adds five.

Disjoint seeds make the pooled test a test. From the Muon-recipe test on, each machine has its own seeds and the pooled McNemar is the primary claim. The window-gate test’s G1 is the case this was for: not shown on X alone, shown on L and pooled.

Audit the records before the next screen. The window gate had been screened, had won, and had been dropped for a reason unrelated to the obstacle at hand; one reading of the records found it. A fixed prediction before the rerun made the audit falsifiable.

Control for the bundled action. A split that copies and resets needed a reset-only arm; so does every intervention with two parts.

Score routing at the write positions. A stream’s channel at key positions can differ from its channel at value positions in a bound run (session H: 34 of the balance-loss arm’s 36 bound-routed runs, and two to five runs per arm elsewhere); the writes are at the values.

Borrowed devices need the model’s read path. Gumbel noise works for a dense read and fails for a routed one; a balance loss balances whatever is cheapest to balance.

15 Limitations

The task. Synthetic, with explicit context tokens; in the grouped layout each key’s stream token is within the window gate’s reach. Distant cues are unresolved, here and in the literature we read.

The recipe needs task structure. The split reads the key positions (as the hinge did). A language model has no marked keys; the analogue is untested.

The model. Three layers (two to six screened), one head, $d = 32$ (to 128 screened), $N = 256$ (to 1024 screened), sequences of at most 99 tokens, a fixed memory half-life of about 14 tokens.

Two streams. The hardest configuration relative to its size; every arm is at 65–69/80 and key splits remain.

Screens. Exploratory sessions use 10–40 runs per arm and lenient rules; their readings are labelled, and nothing from them is a claim until a main-line test repeats it.

16 Distance to a language model

What Phase VI adds to the case. The gate is now a convolution: parallel, cheap, the form every modern linear-attention model already uses. The recipe is two learning-rate groups and a plateau rule. It finds an eight-way partition without labels, and its constants hold to four times the width and twice the depth.

What stands in the way, in order:

1. *Context cues at a distance.* Open in this project and in the literature. The next steps are a probe of where the cue is in the residual stream, a register that overwrites on the cue and holds otherwise, and a per-layer gate over the residual stream (audit, Section 3.3; docs/reading/, Section 4). The risk is that an end-of-sequence loss gives too little pressure to discover the cue label-free; language supplies a denser loss.
2. *The split’s nuisance variable.* The key positions come from the task.
3. *Memory horizon.* A fixed decay; session F’s learned forget gates are the first step, and at eight streams the memory rule that carries them is slower.
4. *Baselines.* Mixture-of-Memories, Raven and Gated DeltaNet route or forget at scale; the comparison that matters is whether their routers discover *contexts* rather than token types, which none of their papers tests.

A staged path. (i) On CPUs: the distant-cue task, the premise test (one delta channel at four and eight streams), eight streams at the larger widths. (ii) A GPU port as a small package, with a chunkwise delta memory, the recall benchmarks [18] and the baselines above, on disjoint seeds. (iii) Small language models, asking one question: do the channels specialize by context or by token type.

17 Conclusion and next steps

Multi-channel Hebbian plasticity in multilayer BDH solves context-conditional binding by partitioning memory, and a label-free gate finds the partition at eight streams: a window gate, slow memory for 2400 updates, and a channel split when binding stalls, in 38 of 40 runs on two machines with disjoint seeds, where the previous recipe bound none. In every look so far the split’s row copy, not its reset, is what works. The obstacle Revision 7 named, the gate’s recurrence, was removed rather than fixed.

Next:

- A pre-registered “copy, not reset” claim and the classifier fix, in the next main-line test.
- Session G’s probe and register on the header layout; session F’s premise test; batch 19’s eight-stream width runs.
- Revision 9 after those, then the GPU package.

A Provenance

test	X: test commit / result commit	L: results file (commit recorded)	runs, X; L
muon_recipe	74f5907 / 3e2d171; pooled 2a6ba3d	L/muon_recipe_results.json (74f5907)	156; 166
window_gate	spec 9a769c7, amended d05f0ca; test 2f98f06, Part D 886a668 / 628eb7c	L/window_gate_results.json (886a668; recorded 1cd20a1)	231; 231

Table 5: Phase VI main-line tests on branch `claude/bdh-growth-hebbian-inference-w90069`. L’s results files for the Phase V tests are now in `results/L/` (00ac0f2, d0e6c85; the Phase IV scale-axes file at 42af095). X: Intel Xeon at 2.80 GHz for both tests; L: Intel i7-12650H; PyTorch 2.14.0.

batch or session	screens	code commit	runs
15 (E)	S41 gate cap, S42 W_h spectrum	90c3a28	45
16 (E)	S43–S44 window gate + slow memory, S45 reservoir, S46 knobs, S47 decoders	93bdfe6 (verdicts 092f355)	165
17 (E)	S48 split on the window gate, S49 key splits	2531480 (verdicts 992e320)	49
18 (E)	S50 reset control and Muon, S51 split at $k=4$, S52 key splits	7ef4e15 (verdicts ac2d3ba)	120
19 (E, running)	S64 width, S65 depth; S66 eight streams and S67 capacity under way	8cff21e	116 so far
H (<code>claude/explore-H</code>)	S58 anti-collapse, S58b at eight streams, S60 temperature	branch head	318
F (<code>claude/explore-F</code>)	delta memory, S53, S54, S59	branch head	322

Table 6: Exploratory work in Phase VI, one thread per run, each segment opened by a bit-for-bit reproduction of a recorded run. The audit is `docs/audit_2026-10.md` (b22da04); the reading notes and synthesis are `docs/reading/` (ad8a92e).

References

- [1] A. Kosowski, P. Uznański, J. Chorowski, Z. Stamirowska, M. Bartoszkiewicz. *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain*. arXiv:2509.26507, 2025.
- [2] B. Engdahl, A. Kosowski, J. Chorowski, Z. Stamirowska, P. Uznański, J. Jiang, R. Phadke, R. Kinas, R. Zhong. *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning*. arXiv:2608.09888, 2026.
- [3] O. Sieberling et al. *Triadic Linear Attention*. arXiv:2609.36529, 2026.
- [4] A. Gu, T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752, 2023.
- [5] K. A. Wang, J. Shi, E. B. Fox. *Test-time regression: a unifying framework for designing sequence models with associative memory*. arXiv:2501.12352, 2025.
- [6] S. Yang, J. Kautz, A. Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. arXiv:2412.06464, 2025.
- [7] A. Jolicoeur-Martineau. *Less is More: Recursive Reasoning with Tiny Networks*. arXiv:2510.04871, 2025.
- [8] J. Geiping et al. *Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach*. arXiv:2502.05171, 2025.

- [9] T. Flesch, D. G. Nagy, A. Saxe, C. Summerfield. *Modelling continual learning in humans with Hebbian context gating and exponentially decaying task signals*. arXiv:2203.11560, 2022.
- [10] A. Iyer et al. *Avoiding Catastrophe: Active Dendrites Enable Multi-Task Learning in Dynamic Environments*. arXiv:2201.00042, 2022.
- [11] M. Choraria et al. *Context-Gated Associative Retrieval: From Theory to Transformers*. arXiv:2605.10970, 2026.
- [12] J. Du et al. *MoM: Linear Sequence Modeling with Mixture-of-Memories*. arXiv:2502.13685, 2025.
- [13] A. Afzal, A. Bick et al. *Raven: High-Recall Sequence Modeling with Sparse Memory Routing*. arXiv:2607.25357, 2026.
- [14] A. Basu. *When Does Content-Based Routing Work? Representation Requirements for Selective Attention in Hybrid Sequence Models*. arXiv:2603.20997, 2026.
- [15] J. Chung, S. Ahn, Y. Bengio. *Hierarchical Multiscale Recurrent Neural Networks*. arXiv:1609.01704, 2017.
- [16] A. Behrouz et al. *Nested Learning: The Illusion of Deep Learning Architectures*. arXiv:2512.24695, 2025.
- [17] W. Fedus, B. Zoph, N. Shazeer. *Switch Transformers*. JMLR 23(120), 2022.
- [18] S. Arora et al. *Zoology: Measuring and Improving Recall in Efficient Language Models*. arXiv:2312.04927, 2023.